

MaleficNet Project Overview Documentation

Structure, Techniques, Libraries, and Execution Workflow

Repository: `mlsec_proj`

Generated on June 6, 2026

Introduction

This repository contains a demonstration implementation of **MaleficNet**, a technique for hiding a binary payload inside the parameters of a deep neural network. The project focuses on a controlled research and educational scenario: it uses a DenseNet model trained on CIFAR-10 and provides the code needed to inject a payload into model weights, retrain or fine-tune the model, extract the payload, and verify that extraction succeeded.

At a high level, the project treats the neural network weights as a noisy communication channel. A payload file is converted into bits, protected with a SHA-256 based integrity check, encoded with LDPC channel coding, spread with pseudo-random spreading codes, and embedded into selected model parameters with a small perturbation controlled by a gain parameter named gamma. The extractor reverses this process by correlating the model weights with the same pseudo-random spreading codes, estimating signal gain and signal-to-noise ratio from a preamble, decoding the LDPC codewords, reconstructing the file, and comparing hashes.

The implementation is intentionally narrow in scope. It supports one main model family, DenseNet, and one dataset, CIFAR-10. This keeps the demo understandable and makes the interaction between the machine-learning pipeline and the information-hiding pipeline easier to inspect.

Safety Scope

The README warns that payloads from malware collections can be live and dangerous. The project should be run only with benign dummy payloads unless it is inside a properly isolated malware-analysis environment. The code writes extracted payloads with the suffix `.no_execute`, but that suffix is only a convention. It does not neutralize harmful content. For routine testing, a small text or binary dummy file is the appropriate payload.

Repository Structure

The tracked project is organized into a small set of top-level scripts and Python packages:

- README.md - Main project description, warnings, dependency notes, and basic command-line usage.
- RUN_SAMPLE.md - Notes from a sample run, including a dummy payload workflow and verification instructions.
- SETUP_MAC_PROM4_REPORT.md - Environment and compatibility report for running the project on macOS Apple Silicon with PyTorch MPS.
- requirements.txt - Pinned Python dependencies used by the current setup.
- maleficnet.py - Main command-line entry point. It wires together the dataset, model, injector, extractor, trainer, device selection, checkpoints, and run modes.
- injector.py - Implements payload preparation and spread-spectrum injection into model weights.
- extractor.py - Implements signal recovery, LDPC decoding, payload reconstruction, and hash verification.
- extractor_callback.py - Defines a PyTorch Lightning callback that periodically attempts extraction during fine-tuning.
- models/densenet.py - DenseNet-121 based PyTorch Lightning module for classification.
- dataset/cifar10.py - PyTorch Lightning data module for CIFAR-10 download, split, and dataloaders.
- utils/utils_bit.py - Bit and byte conversion helpers, plus floating-point bit utilities.
- logger/csv_logger.py - Minimal CSV logger compatible with the PyTorch Lightning logger interface, although the main script currently prefers TensorBoardLogger when available.
- LICENSE - MIT license.

The repository also produces runtime artifacts that are not the conceptual source of the project:

- data/: CIFAR-10 dataset files downloaded by torchvision.

- checkpoints/: pre-injection and post-injection model weights saved by `maleficnet.py`.
- payload/: location where input payloads are expected.
- payload/extract/: location where extracted payloads are written.
- logs/, lightning_logs/, and csvlogger/: training logs and experiment output directories.
- maleficnet.log: application log produced by the main script.

Execution Entry Point

maleficnet.py

`maleficnet.py` is the orchestrator. It performs device detection, parses command-line arguments, initializes the data module and model, prepares the injector and extractor, trains or loads a model, injects the payload, retrains after injection, and finally checks whether extraction succeeds.

Device Selection

The script chooses a compute device using this priority:

- 1 CUDA, when `torch.cuda.is_available()` is true.
- 2 Apple Silicon MPS, when `torch.backends.mps.is_available()` is true.
- 3 CPU otherwise.

The selected device is used both for PyTorch Lightning trainer configuration and for moving updated tensors back into the model after injection.

Command-Line Arguments

The script accepts the following important arguments:

- -dataset - cifar10 - Dataset name. The current implementation supports CIFAR-10.
- -dim - 32 - Input image dimension, with 32 matching CIFAR-10.
- -model, -m - vgg11 - Model name argument. The implemented model initializer recognizes densenet; practical runs should pass `-model densenet`.
- -num_classes - 10 - Number of classification classes.
- -only_pretrained - false - If set, keeps the pretrained DenseNet classifier instead of replacing it. For CIFAR-10, replacement is normally needed.
- -fine_tuning - false - Switches to a fine-tuning workflow that loads a post-injection checkpoint and uses the extraction callback.
- -epochs - 60 - Number of epochs for training or fine-tuning.
- -batch_size - 64 - DataLoader batch size.
- -random_seed - 8 - Seed passed to `torch.manual_seed`.
- -num_workers - 20 - Number of worker processes for data loading.
- -payload - payload.exe - Payload filename expected under `payload/`.
- -gamma - 0.0009 - Injection strength used to perturb model weights.

Model Checkpoints

The main script stores model states under `checkpoints/`. It builds two checkpoint names:

- `{{model}}_{{dataset}}_pre_model.pt`: model state before payload injection.
- `{{model}}_{{dataset}}_{{payload}}_model.pt`: model state after payload injection and retraining.

If the pre-injection checkpoint already exists, the script loads it instead of repeating initial training. This makes repeated experiments faster.

Normal Injection Workflow

When `-fine_tuning` is not set, the workflow is:

- 1 Create the checkpoint directory.
- 2 Initialize the CIFAR-10 data module.
- 3 Initialize the DenseNet model.
- 4 Initialize model weights for linear layers using `weights_init_normal`.
- 5 Initialize Injector with the payload path, output path, seed, device, and chunk factor.
- 6 Initialize Extractor with the same seed, output path, payload length, hash length, and chunk factor.
- 7 Compute the encoded message length that will be embedded.
- 8 Train and test the clean model, unless a clean checkpoint already exists.
- 9 Test the clean model.
- 10 Inject the encoded payload into the model state dictionary.
- 11 Load the injected state dictionary into the model.
- 12 Retrain the model for a few epochs to restore classification performance.
- 13 Test the post-injection model.
- 14 Save the post-injection checkpoint.
- 15 Extract the payload from the final model and verify integrity.

Fine-Tuning Workflow

When `-fine_tuning` is set, the script loads a post-injection checkpoint and uses `ExtractorCallback`. The callback attempts extraction every fixed number of epochs. This mode is useful for observing whether a payload remains recoverable while the model undergoes additional training.

Machine-Learning Components

DenseNet Model

The model wrapper in `models/densenet.py` is a PyTorch Lightning module named `DenseNet`. It uses `torchvision.models.densenet121(pretrained=True)` as the base architecture.

For CIFAR-10 experiments, the final classifier is replaced with a new `nn.Linear` layer whose output dimension matches `num_classes`. This adapts the ImageNet-pretrained backbone to the target classification task.

Forward Pass and Loss

The forward method applies the DenseNet model and then applies `F.log_softmax` over the class dimension. Training, validation, and test steps use negative log-likelihood loss via `F.nll_loss`. This pairing is consistent: `log_softmax` produces log probabilities, and `nll_loss` consumes log probabilities.

Metrics

The model uses `torchmetrics.functional.classification.accuracy` with `task='multiclass'` and `num_classes=self.num_classes`. Metrics logged include:

- `train_loss`, `train_acc`
- `val_loss`, `val_acc`
- `test_loss`, `test_acc`

Optimizer

The optimizer is Adam with a default learning rate of $2e-4$. This is configured in `configure_optimizers`.

CIFAR-10 Data Module

`dataset/cifar10.py` defines a PyTorch Lightning `LightningDataModule`. Its responsibilities are:

- Download CIFAR-10 training and test sets into `data/`.
- Convert images to tensors using `transforms.ToTensor()`.
- Split the training set into 40,000 training samples and 10,000 validation samples.
- Expose train, validation, and test dataloaders.

The data module keeps preprocessing intentionally simple. There is no augmentation or normalization pipeline beyond conversion to tensor. This is appropriate for a compact demo, although stronger augmentation and normalization would normally be considered in a production-grade classifier experiment.

Payload Encoding and Injection

Injector Overview

`injector.py` defines the `Injector` class. It is responsible for converting an input file into a protected bitstream and adding that bitstream to model parameters as a spread-spectrum signal.

The injector is initialized with:

- A deterministic seed, used to generate LDPC matrices, preamble, spreading codes, and selected weight indexes.
- A device, used when writing modified tensors back into the model state dictionary.
- A `malware_path`, which points to the payload file under `payload/`.
- A `result_path`, which is shared with the extractor.
- A logger.
- A `chunk_factor`, which controls how many model weights are used per embedded bit chunk.

Bit Conversion

The payload is loaded with `bits_from_file` from `utils/utils_bit.py`. This function reads bytes from the payload and expands every byte into eight bits, most significant bit first.

For example, a byte is processed by iterating over bit positions 7 down to 0:

```
for i in reversed(range(0, 8)):
    bits.append((b >> i) & 1)
```

The inverse operation is performed by `bits_to_file`, which groups bits back into bytes and writes the recovered file.

Integrity Hash

After reading the payload bits, the injector computes a SHA-256 hash over the string representation of the payload bit sequence. The hexadecimal hash string is encoded as bytes and converted into bits. The final message is:

```
message = payload_bits + hash_bits
```

The extractor later recomputes the hash from recovered payload bits and compares it with the recovered hash bits. This gives the demo a clear success/failure signal after extraction.

LDPC Channel Coding

The project uses `pyldpc.make_ldpc` to build an LDPC parity-check matrix H and generator matrix G . The chosen message block size depends on payload length:

- If the message length is greater than 4000 bits, $k = 3048$.
- Otherwise, $k = 96$.

The code uses variable degree $d_v = 3$ and check degree $d_c = 12$. The encoded block length is computed as:

```
n = k * int(d_c / d_v)
```

Because $d_c / d_v = 4$, this creates an encoded block four times larger than the message block. This redundancy helps the extractor recover bits despite weight noise and subsequent model training.

Chunking and Padding

The message is split into chunks of length k . Full chunks are encoded directly. The final partial chunk is padded with zeros up to length k before encoding. This ensures every LDPC input block has the expected size.

Preamble

Before the LDPC-encoded payload bits, the injector prepends a 200-symbol pseudo-random preamble generated with:

```
np.random.seed(self.seed * 15)
preamble = np.sign(np.random.uniform(-1, 1, 200))
```

The extractor regenerates the same preamble and uses it to estimate signal gain and noise. This is important because the extractor needs a scale estimate before LDPC decoding.

Spread-Spectrum Embedding

The full signal to embed is the concatenation of:

```
b = preamble || LDPC_encoded_message
```

Each symbol in b is embedded by generating a pseudo-random spreading code of values -1 and 1 . The code length is:

```
CHUNK_SIZE * chunk_factor
```

With the repository defaults, $CHUNK_SIZE = 4096$ and $chunk_factor = 6$, so each embedded bit affects 24,576 selected positions.

The perturbation for one embedded symbol is:

```
gamma * spreading_code * bit
```

This perturbation is added to selected flattened model weights. The γ parameter controls the tradeoff between recoverability and model distortion:

- Larger γ : stronger embedded signal and easier extraction, but more risk of degrading model performance.

- Smaller gamma: less visible model perturbation, but extraction may become less reliable.

Weight Selection

The injector flattens all weight tensors except the final weight layer:

```
layers = [n for n in model_st_dict.keys() if "weight" in str(n)][:-1]
```

It records each layer's flattened length, concatenates the weights into one NumPy array, modifies selected positions, and then reconstructs each layer tensor with its original shape.

Selected positions are generated with a deterministic pseudo-random index sequence:

```
np.random.seed(self.seed)
filter_indexes = np.random.randint(
    0,
    len(models_w),
    self.CHUNK_SIZE * self.chunk_factor * number_of_chunks,
    np.int32
).tolist()
```

The same seed and selection logic are used during extraction, so the extractor knows which weights to read without storing the index list separately.

Capacity Check

Before injection, the code checks whether the model has enough available weights for the spreading codes:

```
CHUNK_SIZE * chunk_factor * number_of_chunks <= len(models_w)
```

If the required spreading code footprint is larger than the flattened model weight vector, injection fails and a critical log message is emitted.

Payload Extraction

Extractor Overview

`extractor.py` defines the `Extractor` class. It mirrors the injector: it uses the same seed, LDPC construction parameters, chunk factor, and spreading-code generation. Its job is to recover the embedded signal from model weights and then reconstruct the original payload.

Signal Recovery

The extractor flattens the same model weight layers used by the injector. For each embedded symbol position, it regenerates the corresponding spreading code and selected index range. It then computes a correlation:

```
y_i = spreading_code.T * selected_weights
```

The resulting sequence `y` is an estimate of the embedded preamble plus encoded message.

Gain and SNR Estimation

The first 200 recovered symbols correspond to the known preamble. The extractor computes:

- `gain`: average alignment between recovered preamble symbols and expected preamble symbols.
- `sigma`: normalized standard deviation of the preamble estimate.
- `snr`: $-20 * \log_{10}(\sigma)$.

The recovered LDPC codeword chunks are divided by the gain estimate before decoding. The estimated SNR is passed to `pyldpc.decode`.

LDPC Decoding

After removing the preamble, the extractor divides the remaining signal into LDPC codeword chunks. It decodes each chunk with:

```
decode(self.H, x, snr)
```

The decoded codewords are then transformed back into message bits with `get_message(self.G, x)`.

The project uses Python multiprocessing to parallelize `get_message` calls. It creates a pool with `max(1, mp.cpu_count() - 3)` workers, reserving a few cores for the operating system and other work.

File Reconstruction and Verification

The extractor writes the recovered payload bits to:

```
payload/extract/.no_execute
```

It then splits the decoded message into:

- payload bits, using the known original payload length;
- hash bits, using the known original hash length.

The payload bits are hashed again with SHA-256 using the same bit-string representation used by the injector. The result is converted to bits and compared with the extracted hash bits. Extraction succeeds only if the hashes match.

Fine-Tuning Callback

`extractor_callback.py` contains `ExtractorCallback`, a PyTorch Lightning callback. It stores an epoch counter and calls:

```
self.extractor.extract(pl_module, self.message_length, self.payload)
```

at the end of selected training epochs. The current callback interval is controlled by the `when` argument. In `maleficnet.py`, the callback is created with `when=5`, so extraction is attempted every five callback epochs according to the callback's internal counter.

This callback helps evaluate payload persistence during model fine-tuning. If extraction remains successful after additional epochs, the embedded signal is robust to that amount of training noise.

Logging and Runtime Artifacts

Application Logging

`maleficnet.py` configures Python logging to write to `maleficnet.log`. The injector and extractor log timing, signal-to-noise estimates, hash comparison values, and success/failure status.

Training Logging

The main script attempts to use PyTorch Lightning's `TensorBoardLogger`. If that import fails, it sets the trainer logger to `None`. The repository also contains `logger/csv_logger.py`, a minimal logger that can write training and validation metrics to CSV files. This logger implements methods and properties expected by Lightning, including `log_metrics`, `log_hyperparams`, `name`, `version`, `experiment`, `finalize`, `log_graph`, `log_dir`, and `save`.

Generated Files

Common generated files and directories include:

- checkpoints/*.pt: serialized model state dictionaries.
- payload/extract/*.no_execute: extracted payload reconstructions.
- maleficnet.log: injection and extraction log messages.
- logs/ or lightning_logs/: Lightning experiment logs.
- data/cifar-10-batches-py/: downloaded CIFAR-10 data.

Libraries and Dependencies

The pinned dependencies in requirements.txt are:

- torch - 2.11.0 - Core tensor library, neural network execution, optimizer, serialization, CUDA/MPS/CPU device support.
- torchvision - 0.26.0 - Provides CIFAR-10 dataset utilities, transforms, and pretrained DenseNet-121.
- pytorch-lightning - 2.6.1 - Structures the model, data module, training loop, testing loop, callbacks, and logging.
- pyldpc - 0.7.9 - Builds LDPC matrices and performs channel encoding/decoding for robust payload recovery.
- bitstring - 4.4.0 - Used by bit utility helpers for inspecting floating-point bytes.
- torchmetrics - 1.9.0 - Computes classification accuracy during training, validation, and testing.
- numpy - 2.4.4 - Provides vectorized array manipulation, random generation, flattening, and correlation computations.
- scipy - 1.17.1 - Scientific dependency required by LDPC and numerical tooling.

The standard library is also important:

- argparse: command-line interface.
- hashlib: SHA-256 integrity hash.
- logging: runtime logs.
- math: chunk count computation.
- multiprocessing: parallel extraction post-processing.
- pathlib: platform-independent filesystem paths.
- struct: floating-point byte conversion helpers.
- time: injection and extraction timing.
- warnings: suppression of noisy image warnings.

Technique Summary

Spread-Spectrum Channel Coding

The central idea is to hide information by spreading each payload bit across many model parameters. Instead of changing one weight per bit, the injector adds a small pseudo-random pattern across thousands of weights. The extractor can recover the bit by correlating the same pseudo-random pattern with the modified weights.

This resembles code-division multiple access and spread-spectrum communication: the embedded message is distributed over a high-dimensional carrier, and only someone who knows the seed and spreading process can efficiently reconstruct the signal.

Why LDPC is Used

Model weights are noisy carriers. Training, fine-tuning, optimizer updates, and the original distribution of weights all interfere with the embedded signal. LDPC coding adds redundancy before embedding, allowing the extractor to correct some errors after recovering a noisy sequence.

Why a Preamble is Used

The extractor does not directly know the effective scale of the recovered signal. The preamble is a known sequence placed before the encoded payload. By comparing recovered preamble values to the expected preamble, the extractor estimates gain and SNR, which improves LDPC decoding.

Why Retraining is Performed

Payload injection perturbs model weights. Retraining after injection is intended to restore model accuracy while preserving enough embedded signal for extraction. This creates a tension between task performance and payload recoverability, which is the central experimental tradeoff of the demo.

End-to-End Data Flow

The normal run can be summarized as:

- 1 User places a payload file under `payload/`.
- 2 `maleficnet.py` initializes the data module and DenseNet model.
- 3 The model is trained or loaded from a clean checkpoint.
- 4 Injector reads the payload and converts it to bits.
- 5 Injector appends a SHA-256 hash represented as bits.
- 6 Injector LDPC-encodes the message and prepends a preamble.
- 7 Injector embeds the resulting signal into flattened model weights.
- 8 The injected model is retrained and saved.
- 9 Extractor reads the same weight positions using the same seed.
- 10 Extractor estimates the embedded symbols via correlation.
- 11 Extractor uses the preamble to estimate gain and SNR.
- 12 Extractor decodes LDPC chunks and rebuilds the payload.
- 13 Extractor writes the recovered file and verifies the recovered hash.

Reproducible Usage

A safe smoke test should use a benign dummy payload:

```
mkdir -p payload
printf "test" > payload/dummy.bin
./venv/bin/python maleficnet.py \
  --epochs 1 \
  --model densenet \
  --payload dummy.bin \
  --num_workers 2
```

After the run, verify extraction by comparing checksums:

```
shasum -a 256 payload/dummy.bin payload/extract/dummy.bin.no_execute
```

Matching hashes indicate that the extracted payload bytes match the original dummy payload.

Important Implementation Details and Caveats

Supported Model Argument

The argument parser defaults `-model` to `vgg11`, but `initialize_model` only creates a model when `model_name == "densenet"`. Therefore, practical commands should explicitly pass `-model densenet`.

Payload Size and Model Capacity

Large payloads require more encoded bits, more spreading chunks, and therefore more model parameters. If the spreading footprint exceeds available weights, the injector and extractor report that spreading codes cannot be bigger than the model.

Layer Selection

The code embeds into all flattened weight tensors except the last weight layer. Excluding the final layer can reduce direct disturbance to the classifier head, but the exact impact depends on the architecture and task.

Determinism

The injector and extractor rely on matching seeds. If the seed, chunk factor, layer selection, or model architecture changes between injection and extraction, recovery will likely fail.

Compatibility Notes

The current codebase reflects updates for modern PyTorch Lightning and torchmetrics APIs, plus Apple MPS support. The setup report records observed versions and a successful smoke-test environment.

Extension Points

Natural extensions include:

- Adding more model architectures in `initialize_model`.
- Adding more datasets by implementing additional Lightning data modules.
- Experimenting with payload size, `gamma`, `chunk_factor`, and LDPC parameters.
- Implementing a model-weight analyzer to choose lower-risk weights rather than selecting positions uniformly at random.
- Adding structured experiment outputs for accuracy, extraction success, SNR, payload size, and timing.
- Adding automated tests for bit conversion, hash reconstruction, chunk padding, and small synthetic injection/extraction cases.

Conclusion

The project is a compact end-to-end demonstration of hiding a binary payload in neural-network parameters using spread-spectrum techniques and LDPC channel coding. The source tree separates the workflow cleanly: `maleficnet.py` orchestrates the run, `models/` and `dataset/` define the learning task, `injector.py` implements embedding, `extractor.py` implements recovery, and `utils/` provides low-level bit operations. The main technical tradeoff is between maintaining model performance and keeping the embedded signal recoverable after training noise.

Because the repository is about malware-hiding research, it should be handled with care. For development and demonstration, benign dummy payloads provide the same technical coverage without introducing operational risk.